PRODUCT REQUIREMENTS DOCUMENT

BACKEND SYSTEM

Interactive Audio Learning Platform for Visually Impaired Students

Document Type	Product Requirements Document (PRD)
System Layer	Full Backend
Version	1.0
Date	March 2026
Status	APPROVED FOR DEVELOPMENT
Audience	Backend Developer(s)
Project	VisioLearn — Interactive Audio Learning Platform

CONFIDENTIAL — INTERNAL DEVELOPMENT USE ONLY

1. Executive Summary

This document is the authoritative Product Requirements Document (PRD) for the complete backend system of VisioLearn — an offline-first, interactive audio learning platform designed to empower visually impaired students in Uganda. The backend serves as the central intelligence and data hub of the platform, powering teacher content management, AI-driven question and summary generation (content-bound RAG), student progress synchronisation, push notifications, analytics, and administration.

The backend must be engineered for maximum performance given the constraint that end devices are basic Android phones operating predominantly offline in low-bandwidth environments. Every API response must be optimised for minimum latency, minimum payload size, and graceful degradation when connectivity is unstable.

The system is built with Python FastAPI on PostgreSQL. It exposes a RESTful API consumed by the Android client, and an administrative web interface for school administrators and research staff. All AI processing happens server-side, and AI-generated artefacts (questions, summaries, feedback) are pushed to the device for completely offline student use.

Attribute	Value
Project Name	VisioLearn — Interactive Audio Learning Platform
Backend Stack	Python (FastAPI) + PostgreSQL + Redis + Celery
Primary Client	Android (Kotlin) — API Level 26+
Connectivity Model	Offline-first; sync on reconnect
AI Architecture	Content-bound RAG (server-side) — no hallucinations
Deployment Target	Cloud VPS (initially free tier; scalable)
Target Users	Visually impaired students & teachers in Uganda
Pilot Scale	30–50 students, 5–10 teachers, 2–3 schools

2. Document Conventions & Terminology

Term / Acronym	Definition
VisioLearn	The product name for the entire platform
PRD	Product Requirements Document

Backend	Python FastAPI server, database, cache, AI pipeline, workers
Android Client	The Kotlin Android app consumed by students and teachers
Teacher	A registered educator who uploads lesson notes
Student	A registered visually impaired learner who studies uploaded content
Lesson Note	A document (.txt, .docx, .pdf) uploaded by a teacher for a topic
Learning Unit	A structured chunk of a Lesson Note segmented for audio delivery
AI Artefact	Questions, summaries, or feedback generated by the AI pipeline
Local DB	Room database on the Android device (SQLite)
Sync	The process of uploading offline progress and downloading new artefacts when internet is available
RAG	Retrieval-Augmented Generation — AI answers grounded strictly in teacher-uploaded text
Constrained AI	AI module that ONLY uses teacher-uploaded text; never external knowledge
JWT	JSON Web Token — used for stateless API authentication
SRS	SHALL = mandatory; SHOULD = strongly recommended; MAY = optional

3. Project Context & Problem Being Solved

3.1 Research Background

Visually impaired students in Uganda are passive learners. Existing tools (Blind Assistant, Yee FM, JAWS) provide audio access to text but offer zero interaction: no questions, no feedback, no progress tracking. Under Uganda's new competency-based curriculum, this gap is academically fatal — 112 visually impaired candidates appeared in the 2026 UACE with disproportionately low pass rates.

VisioLearn transforms the paradigm. Teachers upload lesson notes once; the backend processes them, generates curriculum-aligned questions and summaries through constrained AI, and pushes the artefacts to students' devices for completely offline learning. Students listen, answer, receive feedback, and accumulate tracked progress — all without internet.

3.2 The Backend's Role in the Platform

The Android client is entirely offline during student learning sessions. The backend is engaged in two windows:

- Teacher window: when a teacher has internet connectivity to upload a note.
- Sync window: when a student or teacher briefly connects to internet to sync progress up and pull new artefacts down.

Everything the student experiences during learning — audio content, questions, answer feedback, progress records — originates from the backend but lives on-device. The backend must therefore process notes thoroughly, generate high-quality artefacts, and deliver them efficiently during the brief sync windows available on limited data connections.

3.3 Non-Negotiable Constraints

- All AI generation MUST be strictly content-bound (no external knowledge sources).
- All generated artefacts MUST be storable on-device for fully offline use.
- API responses MUST be lean and fast — target devices are low-end Android phones.
- The backend MUST handle intermittent sync — partial uploads, retry, resumption.
- The system MUST be deployable on a free or low-cost cloud tier for the pilot phase.
- All personal data MUST be handled in compliance with applicable data protection norms.

4. System Architecture Overview

4.1 High-Level Architecture

The backend is structured as a layered, asynchronous system composed of the following components:

Component	Responsibility
FastAPI Application Server	RESTful API layer; handles all HTTP requests from Android clients and admin panel; input validation, authentication, routing
PostgreSQL Database	Primary persistent store for all entities: users, schools, notes, learning units, AI artefacts, progress records, analytics events
Redis Cache	Session tokens, rate-limit counters, hot-path response caching, Celery broker message queue
Celery Worker Pool	Asynchronous task processing: document parsing, text chunking, AI generation pipeline, sync batch processing, notification dispatch
AI Processing Pipeline	Content-bound NLP engine: text extraction → chunking → question generation → summary generation → feedback templates; strictly constrained to uploaded text
File Storage	Secure blob storage for uploaded raw note files (local filesystem for pilot; S3-compatible for production)
Admin Interface	Lightweight internal dashboard for school admin, content moderation, analytics review (FastAPI + Jinja2 or separate lightweight SPA)
Background Scheduler	Celery Beat: scheduled jobs for analytics aggregation, stale session cleanup, sync health checks

4.2 Technology Stack Specification

Layer	Technology & Justification
Runtime	Python 3.11+ — FastAPI requires Python 3.8+; 3.11 delivers 25% speed improvement via optimised bytecode
Web Framework	FastAPI — async-native, automatic OpenAPI docs, Pydantic v2 validation, superior performance vs Django REST or Flask
ASGI Server	Uvicorn with Gunicorn workers — production-grade ASGI; multi-worker process model for concurrency under sync bursts
ORM	SQLAlchemy 2.0 (async) + Alembic migrations — type-safe, async queries, full migration history
Primary Database	PostgreSQL 15+ — ACID compliance, JSONB for flexible AI artefact storage, full-text search, excellent Python driver ecosystem
Cache / Broker	Redis 7+ — in-memory speed for hot paths; doubles as Celery message broker; sorted sets for rate limiting
Task Queue	Celery 5 — distributed async worker pool; priority queues; retry with exponential backoff; ideal for AI pipeline
AI / NLP	spaCy (en_core_web_sm, local) + custom rule-based question generation; sentence-transformers for semantic similarity; all models run server-side
Document Parsing	python-docx (Word), PDFMiner.six (PDF text extraction), plain text readers — mirrors the Android Apache POI approach but server-side
Text-to-Speech Prep	Server returns structured text to client; client uses Android TTS — backend stores text chunks, not audio files (saves bandwidth)
Authentication	JWT (python-jose) with refresh token rotation; bcrypt password hashing
File Storage (Pilot)	Local filesystem with SHA-256 content addressing; path: /data/notes/<school_id>/<note_id>.<ext>
File Storage (Scale)	S3-compatible (Backblaze B2 / AWS S3) — switchable via environment config
Containerisation	Docker + Docker Compose for local dev; Dockerfile for cloud deployment
API Documentation	Auto-generated OpenAPI 3.0 via FastAPI — accessible at /docs and /redoc
Testing	pytest + pytest-asyncio + httpx AsyncClient; coverage target 80%+
CI/CD	GitHub Actions — lint (ruff), type-check (mypy), test, build, deploy

4.3 Data Flow: Note Upload to Offline Artefact

The following describes the complete data flow from a teacher uploading a note to a student using AI artefacts fully offline:

1. Teacher uploads note file via POST /api/v1/notes with file attachment and metadata (subject, topic, grade level).
2. FastAPI validates file type, size, and user permissions. File is saved to blob storage. Note record created in DB with status PENDING_PROCESSING.
3. A Celery task (process_note) is enqueued with high priority.
4. Celery worker picks up the task: (a) extracts raw text from file, (b) segments into Learning Units (paragraphs/sections), (c) stores units in DB.
5. A follow-on task (generate_artefacts) runs the AI pipeline per Learning Unit: generates questions (MCQ + short answer), a summary, and feedback templates — ALL strictly from the unit's text.
6. AI artefacts are stored in DB linked to their Learning Unit. Note status updated to READY.
7. Next time the student's Android app syncs, GET /api/v1/sync returns a payload containing all new/updated Learning Units and AI Artefacts since the device's last_synced_at timestamp.
8. Android client stores everything in Room DB. Student now learns entirely offline.
9. Student answers questions offline; responses are stored in Room DB.
10. On next sync, Android client POSTs student progress (answers, scores, completion) to POST /api/v1/sync/progress. Backend stores records in PostgreSQL for analytics.

5. Data Models & Database Schema

All tables use PostgreSQL. UUID primary keys throughout for security and scalability. All timestamps are stored in UTC. Soft-delete pattern used on all critical entities (is_deleted flag + deleted_at timestamp).

5.1 Entity: schools

Column	Type & Notes
id	UUID PK DEFAULT gen_random_uuid()
name	VARCHAR(255) NOT NULL — full school name
district	VARCHAR(100) — Ugandan district
region	VARCHAR(100) — e.g. Central, Northern
contact_email	VARCHAR(255)
is_active	BOOLEAN DEFAULT TRUE
created_at	TIMESTAMPTZ DEFAULT now()
updated_at	TIMESTAMPTZ DEFAULT now()

5.2 Entity: users

Column	Type & Notes
id	UUID PK
school_id	UUID FK -> schools.id NOT NULL
role	ENUM('student','teacher','school_admin','system_admin') NOT NULL
username	VARCHAR(80) UNIQUE NOT NULL
email	VARCHAR(255) UNIQUE — nullable for students without email
password_hash	VARCHAR(255) NOT NULL — bcrypt
display_name	VARCHAR(120)
grade_level	VARCHAR(20) — for students (e.g. S4, P6)
subjects	JSONB — for teachers: array of subjects they teach
is_active	BOOLEAN DEFAULT TRUE
last_login_at	TIMESTAMPTZ
created_at	TIMESTAMPTZ DEFAULT now()
updated_at	TIMESTAMPTZ DEFAULT now()
is_deleted	BOOLEAN DEFAULT FALSE
deleted_at	TIMESTAMPTZ

5.3 Entity: refresh_tokens

Column	Type & Notes
id	UUID PK
user_id	UUID FK -> users.id NOT NULL
token_hash	VARCHAR(255) NOT NULL — SHA-256 of the token string
expires_at	TIMESTAMPTZ NOT NULL
is_revoked	BOOLEAN DEFAULT FALSE
created_at	TIMESTAMPTZ DEFAULT now()
device_fingerprint	VARCHAR(255) — optional Android device ID hash for security

5.4 Entity: lesson_notes

Column	Type & Notes
id	UUID PK
teacher_id	UUID FK -> users.id NOT NULL

school_id	UUID FK -> schools.id NOT NULL
title	VARCHAR(255) NOT NULL
subject	VARCHAR(100) NOT NULL
topic	VARCHAR(255) NOT NULL
grade_level	VARCHAR(20)
curriculum_alignment	VARCHAR(100) — e.g. Uganda CBC S4
file_path	VARCHAR(500) — path in blob storage
file_type	ENUM('txt','docx','pdf') NOT NULL
file_size_bytes	INTEGER
file_hash	VARCHAR(64) — SHA-256; used for deduplication
status	ENUM('uploaded','processing','ready','failed') DEFAULT 'uploaded'
processing_error	TEXT — populated on failure for debugging
word_count	INTEGER — extracted from parsed text
language	VARCHAR(20) DEFAULT 'en'
is_public	BOOLEAN DEFAULT FALSE — if TRUE, visible to all schools
version	INTEGER DEFAULT 1 — for future update support
created_at	TIMESTAMPTZ DEFAULT now()
updated_at	TIMESTAMPTZ DEFAULT now()
is_deleted	BOOLEAN DEFAULT FALSE
deleted_at	TIMESTAMPTZ

Index: CREATE INDEX idx_notes_school_subject ON lesson_notes(school_id, subject);

Index: CREATE INDEX idx_notes_status ON lesson_notes(status) WHERE status = 'uploaded' OR status = 'processing';

5.5 Entity: learning_units

Column	Type & Notes
id	UUID PK
note_id	UUID FK -> lesson_notes.id NOT NULL
sequence_number	INTEGER NOT NULL — order of unit within note
heading	VARCHAR(255) — detected section heading, if any
text_content	TEXT NOT NULL — the actual text chunk delivered to Android TTS
word_count	INTEGER
estimated_audio_seconds	INTEGER — word_count / 2.5 (avg TTS words per second)

created_at	TIMESTAMPTZ DEFAULT now()

Index: CREATE INDEX idx_units_note_seq ON learning_units(note_id, sequence_number);

5.6 Entity: ai_artefacts

Column	Type & Notes
id	UUID PK
unit_id	UUID FK -> learning_units.id NOT NULL
artefact_type	ENUM('mcq_question','short_answer_question','summary','feedback_template') NOT NULL
content	JSONB NOT NULL — see schema below per type
generation_model	VARCHAR(80) — e.g. 'spacy-rule-v1', 'transformers-v1'
confidence_score	FLOAT — 0.0–1.0 quality estimate
is_approved	BOOLEAN DEFAULT TRUE — teacher can flag for review
created_at	TIMESTAMPTZ DEFAULT now()
updated_at	TIMESTAMPTZ DEFAULT now()

JSONB Schema: mcq_question

Field	Description
question_text	The question string
options	Array of 4 option objects: {id, text}
correct_option_id	The id of the correct option (stored securely; NOT sent to student in initial download)
explanation	One-sentence explanation of the correct answer, sourced from the unit text
source_sentence	The verbatim sentence from unit text that grounds this question

JSONB Schema: summary

Field	Description
summary_text	2–5 sentence summary of the learning unit; entirely from unit text
key_terms	Array of up to 5 key vocabulary terms extracted from the unit

5.7 Entity: student_progress

Column	Type & Notes

id	UUID PK
student_id	UUID FK -> users.id NOT NULL
unit_id	UUID FK -> learning_units.id NOT NULL
artefact_id	UUID FK -> ai_artefacts.id — nullable (for unit completion vs question answer)
event_type	ENUM('unit_started','unit_completed','question_answered','note_completed') NOT NULL
student_answer	TEXT — raw answer for short_answer; option_id for MCQ
is_correct	BOOLEAN — NULL for non-question events
score	FLOAT — partial credit for short answers (0.0–1.0)
attempt_number	INTEGER DEFAULT 1
time_spent_seconds	INTEGER — seconds spent on this unit/question
device_id_hash	VARCHAR(64) — anonymised device identifier
synced_at	TIMESTAMPTZ — when this record was synced from device
offline_recorded_at	TIMESTAMPTZ NOT NULL — actual event time on device

Index: CREATE INDEX idx_progress_student_unit ON student_progress(student_id, unit_id);

Index: CREATE INDEX idx_progress_student_time ON student_progress(student_id, offline_recorded_at DESC);

5.8 Entity: note_assignments

Links a note to students or grade groups within a school, giving teachers control over who receives which content.

Column	Type & Notes
id	UUID PK
note_id	UUID FK -> lesson_notes.id NOT NULL
school_id	UUID FK -> schools.id NOT NULL
assigned_to_type	ENUM('grade_level','individual_student','all_school') NOT NULL
grade_level	VARCHAR(20) — used when assigned_to_type='grade_level'
student_id	UUID FK -> users.id — used when assigned_to_type='individual_student'
assigned_by	UUID FK -> users.id NOT NULL — teacher who made the assignment
assigned_at	TIMESTAMPTZ DEFAULT now()
is_active	BOOLEAN DEFAULT TRUE

5.9 Entity: analytics_events

Anonymised aggregate analytics for research and policymakers. No personally identifiable information in this table.

Column	Type & Notes
id	UUID PK
school_id	UUID FK -> schools.id
event_date	DATE NOT NULL
grade_level	VARCHAR(20)
subject	VARCHAR(100)
metric_key	VARCHAR(100) — e.g. avg_completion_rate, avg_score_mcq
metric_value	FLOAT
sample_size	INTEGER
created_at	TIMESTAMPTZ DEFAULT now()

6. REST API Specification

Base URL: /api/v1. All responses return JSON. All authenticated endpoints require Authorization: Bearer <access_token>. All endpoints return standard envelope: { success, data, error, pagination }.

6.1 Authentication Endpoints

Method + Path	Auth	Description
POST /auth/register	None	Register new user (teacher or student). School admin creates accounts for their school. Returns 201 with user object.
POST /auth/login	None	Email/username + password login. Returns access_token (15 min TTL), refresh_token (30 days TTL), and user profile.
POST /auth/refresh	None (refresh token in body)	Rotate refresh token; return new access + refresh token pair. Old refresh token is revoked.
POST /auth/logout	JWT	Revoke the refresh token for the current device session.

POST /auth/change-password	JWT	Change password; revoke ALL refresh tokens for user.
POST /auth/reset-password/request	None	Initiate password reset via email (teacher) or admin-triggered (student without email).
POST /auth/reset-password/confirm	None	Confirm reset with token; set new password.

6.2 User Management Endpoints

Method + Path	Role Required	Description
GET /users/me	Any authenticated	Returns current user's profile.
PATCH /users/me	Any authenticated	Update display name, grade level, subject list.
GET /users	school_admin, system_admin	List users in the requester's school (school_admin) or all schools (system_admin). Supports pagination and filtering by role.
POST /users	school_admin, system_admin	Create a new user account within a school.
PATCH /users/{user_id}	school_admin, system_admin	Update user account (activate/deactivate, change grade, etc.).
DELETE /users/{user_id}	school_admin, system_admin	Soft-delete user. Returns 200 with confirmation.

6.3 School Management Endpoints

Method + Path	Role Required	Description
GET /schools	system_admin	List all registered schools with summary stats.
POST /schools	system_admin	Register a new school.
GET /schools/{school_id}	school_admin (own), system_admin	Get school profile and summary statistics.
PATCH /schools/{school_id}	system_admin	Update school details.

6.4 Lesson Note Endpoints

Method + Path	Role Required	Description
POST /notes	teacher	Upload a lesson note. Multipart form: file + metadata (title, subject, topic, grade_level). Validates file type (txt/docx/pdf), max size 10MB. Enqueues processing task. Returns 202 with note_id and status='uploaded'.
GET /notes	teacher (own notes), school_admin (school notes)	List notes. Filter by status, subject, grade_level. Paginated. Returns summary objects (no file path).
GET /notes/{note_id}	teacher (own), school_admin, student (if assigned)	Get full note metadata including processing status and unit count.
GET /notes/{note_id}/status	teacher (own)	Lightweight polling endpoint: returns {status, unit_count, artefact_count, error_message}. Used by teacher app to track processing.
PATCH /notes/{note_id}	teacher (own)	Update metadata (title, subject, topic) for READY notes only.
DELETE /notes/{note_id}	teacher (own), school_admin	Soft-delete note and all associated units/artefacts. Cascades to note_assignments.
GET /notes/{note_id}/units	teacher (own), school_admin	List all learning units for a note. Used in admin preview.
GET /notes/{note_id}/artefacts	teacher (own), school_admin	List all AI artefacts for a note (grouped by unit). Used for teacher review of generated questions.
PATCH /artefacts/{artefact_id}/approve	teacher (own note)	Toggle is_approved on an artefact. Unapproved artefacts are excluded from student sync.

6.5 Note Assignment Endpoints

Method + Path	Role Required	Description
POST /assignments	teacher	Assign a note to a grade level or individual student within the school. Body: {note_id, assigned_to_type, grade_level?, student_id?}.

GET /assignments	teacher, school_admin	List all active assignments for the school. Filter by note_id or grade_level.
DELETE /assignments/{assignment_id}	teacher (own), school_admin	Deactivate an assignment. Students who already have the content retain it locally.

6.6 Sync Endpoints (Core Offline-First API)

These endpoints are the heartbeat of the offline-first architecture. They are optimised for minimal payload size and bandwidth.

Method + Path	Role Required	Description
GET /sync/pull	student, teacher	Returns all content assigned to the authenticated user that has changed since last_synced_at. Query param: ?since=<ISO8601_timestamp>. Response includes: new/updated notes metadata, learning_units (text chunks), ai_artefacts (WITHOUT correct answers). Uses delta sync — only changed records returned.
POST /sync/push	student	Submit offline-recorded student progress. Body: array of progress event objects. Server validates, deduplicates, and stores. Returns 200 with processed_count and any validation errors.
GET /sync/answers/{unit_id}	student	After a student has answered all questions for a unit, this endpoint releases the correct answers and explanations for that unit. Prevents answer pre-fetching.
GET /sync/status	student, teacher	Returns sync health: server timestamp, pending artefacts count, last successful sync.

Sync Pull Response Schema

Field	Description
server_timestamp	ISO8601 UTC — client must store this as new last_synced_at
notes	Array of note metadata objects assigned to this user since last sync
units	Array of learning unit objects (id, note_id, sequence_number, heading, text_content, estimated_audio_seconds)

artefacts	Array of artefact objects. MCQ objects include question_text and options but NOT correct_option_id. Summary artefacts include full summary_text.
revoked_ids	Array of note/unit IDs that have been deleted — client should remove these from local DB
total_bytes_estimate	Rough byte count of this payload for client progress display

6.7 Progress & Analytics Endpoints

Method + Path	Role Required	Description
GET /progress/me	student	Student's own progress summary: units completed, questions answered, average score, time spent, per-note breakdown.
GET /progress/students	teacher, school_admin	Aggregated progress report for the school/grade. Query params: ?grade_level, ?subject, ?note_id. Returns anonymised aggregate stats (no individual student data exposed to teachers).
GET /analytics/school/{school_id}	school_admin, system_admin	School-level analytics: engagement rates, completion rates, average scores by subject. Date range filter.
GET /analytics/export	system_admin	CSV export of anonymised analytics for research use. Feeds DHIS2 / UCC observatory.

6.8 Admin Endpoints

Method + Path	Role Required	Description
GET /admin/dashboard	system_admin	System-wide health: total schools, users, notes, processing queue depth, error rates.
GET /admin/processing-queue	system_admin	View Celery task queue status and pending jobs.
POST /admin/reprocess/{note_id}	system_admin	Manually trigger reprocessing of a failed note.
GET /admin/errors	system_admin	List of notes in 'failed' status with error messages.

POST /admin/schools/{school_id}/bulk-users	system_admin	Bulk-import users from CSV for a school. Useful for pilot onboarding.

7. AI Processing Pipeline — Content-Bound Architecture

This section defines the complete AI pipeline that runs server-side as Celery tasks after a note is uploaded. The pipeline is architecturally constrained: it MUST ONLY generate questions, summaries, and feedback from the text of the uploaded note. No external knowledge, no internet lookups, no language model hallucinations.

7.1 Stage 1: Document Parsing

Input / Output	Detail
Input	Raw file stored in blob storage (.txt / .docx / .pdf)
Output	Clean, structured plain text string
.txt handling	Read with utf-8 encoding; strip non-printable characters; preserve paragraph structure via double-newlines
.docx handling	python-docx: extract paragraph text preserving heading hierarchy; table cells extracted row-by-row; skip headers/footers
.pdf handling	PDFMiner.six: extract text layer only; text-based PDFs only (scanned/image PDFs rejected at upload validation)
Error handling	If extraction yields < 50 words, mark note as 'failed' with error: 'Insufficient extractable text'. Notify teacher.
Language detection	langdetect to set language field; used for TTS language hint sent to Android client

7.2 Stage 2: Text Chunking into Learning Units

Rule	Detail
Primary split signal	Detected headings (from docx heading styles or PDF heading heuristics)
Secondary split signal	Topic shift detected by sentence-transformers cosine similarity drop below 0.45 threshold
Size constraint	Each unit: minimum 100 words, maximum 600 words. Units above 600 words are split at the nearest sentence boundary.
Short unit merging	Units below 100 words are merged with the following unit unless the following unit would exceed 600 words.

Heading preservation	Each unit stores the closest preceding heading as its heading field. If none detected, heading is NULL.
Sequence numbering	Units numbered 1..N in extraction order; used by Android client for linear audio navigation
Output	Ordered list of {heading, text_content, word_count} objects stored as learning_units rows

7.3 Stage 3: Question Generation

Questions are generated per Learning Unit using a pipeline of NLP techniques. All questions are strictly grounded in the unit text.

MCQ Generation Algorithm

11. Parse unit text with spaCy (en_core_web_sm).
12. Extract candidate fact sentences: sentences containing named entities (PERSON, ORG, GPE, DATE, CARDINAL, PRODUCT) or sentences with subject-verb-object triples.
13. For each candidate sentence, apply cloze transformation: mask the key entity/noun phrase to form the question stem.
14. Generate 3 distractor options using: (a) other entities of the same type from the same unit text, (b) semantically similar terms from sentence-transformers nearest-neighbor search within the unit corpus.
15. Shuffle options; assign random option IDs (A/B/C/D).
16. Store correct_option_id server-side ONLY; excluded from sync-pull payload.
17. Generate 1-sentence explanation by extracting the surrounding context of the source sentence.
18. Target: 3–6 MCQ questions per Learning Unit. Skip generation if unit has fewer than 4 candidate sentences.

Short Answer Question Generation

19. Identify the 2–3 most information-dense sentences in the unit using TF-IDF scoring.
20. Transform each into a 'What / Who / When / How' question by replacing the subject or key predicate.
21. Store the source sentence as the model answer for scoring.
22. Target: 1–2 short answer questions per Learning Unit.

Answer Scoring for Short Answers

23. On sync/answers endpoint call, compare student's text answer against model answer.
24. Compute cosine similarity using sentence-transformers (paraphrase-MiniLM-L6-v2, quantised for server efficiency).
25. Score: >= 0.75 = correct (1.0), 0.50–0.74 = partial (0.5), < 0.50 = incorrect (0.0).
26. Return score and the model answer text for student learning feedback.

7.4 Stage 4: Summary Generation

Rule	Detail
Method	Extractive summarisation — select the 2–4 most representative sentences from the unit using TF-IDF + positional scoring (first and last sentences weighted higher).
Length constraint	Summary MUST be <= 20% of unit word count, minimum 2 sentences.
Key terms extraction	spaCy noun chunks + named entities; deduplicated; top 5 by frequency.
Constraint check	Validate that every sentence in the summary exists verbatim in the source unit text — prevents any hallucination.

7.5 Stage 5: Quality Control & Approval

Check	Rule
Minimum artefact count	If a unit produces 0 questions (e.g. narrative unit with no extractable facts), generate 1 comprehension question from the summary.
Confidence scoring	Each artefact assigned a confidence_score based on: entity type quality, sentence length, syntactic complexity. Low-confidence artefacts (< 0.40) flagged for teacher review (is_approved = FALSE).
Teacher review	Teachers can view all generated artefacts via GET /notes/{note_id}/artefacts and toggle approval via PATCH /artefacts/{id}/approve. Unapproved artefacts are excluded from student sync.
Duplicate detection	Questions with cosine similarity > 0.85 to another question in the same note are deduplicated — keep the higher-confidence one.

8. Authentication & Authorisation

8.1 JWT Strategy

Parameter	Specification
Algorithm	HS256 (HMAC-SHA256); upgrade to RS256 for multi-service scale
Access token TTL	15 minutes — short to minimise breach window
Refresh token TTL	30 days — stored as hash in DB; rotated on each use
Token payload	{sub: user_id, role: role, school_id: school_id, iat, exp}

Refresh rotation	Each refresh token use invalidates the old token and issues a new pair (token family model)
Revocation	Logout revokes the specific device refresh token. Password change revokes ALL refresh tokens for the user.
Storage on device	Android client stores tokens in EncryptedSharedPreferences (Android Keystore)

8.2 Role-Based Access Control (RBAC)

Resource	student	teacher	school_admin	system_admin
Own profile	R/W	R/W	R/W	R/W
Other users (own school)	—	—	R/W	R/W
Users (any school)	—	—	—	R/W
Upload notes	—	CREATE	—	R/W
Manage own notes	—	R/W/D	Read	R/W/D
Assign notes	—	CREATE	R/W	R/W
View own progress	Read	—	—	Read
View school analytics	—	Aggregated	Full	Full
AI artefact approval	—	own notes	own school	All
System admin panel	—	—	—	Full

8.3 Security Requirements

- All endpoints MUST be served over HTTPS. HTTP requests MUST be redirected to HTTPS.
- Passwords MUST be hashed with bcrypt, minimum cost factor 12.
- Login endpoint MUST implement rate limiting: max 10 failed attempts per 15 minutes per IP. Lock account after 20 failed attempts.
- Refresh token MUST be validated against DB hash on every use — not just signature.
- All API inputs MUST be validated by Pydantic models before processing.
- File uploads MUST be validated for MIME type (not just extension), scanned for malformed structure.
- File paths MUST use content-addressed naming (UUID/hash) — no user-controlled file path components.

- SQL queries MUST use parameterised statements via SQLAlchemy ORM — no raw string interpolation.
- Sensitive environment variables (DB URL, JWT secret, storage keys) MUST be stored in environment/.env and NEVER in source code.
- CORS MUST be configured to whitelist only the Android app's declared origins in production.

9. Performance Requirements & Optimisation Strategy

Performance is critical. Students sync over mobile data (often GPRS/EDGE in rural Uganda). Teachers upload over Wi-Fi or 3G. Every millisecond and every byte counts.

9.1 Response Time Targets (P99)

Endpoint Category	Target Latency (P99)
POST /auth/login	< 400ms
POST /auth/refresh	< 150ms
GET /sync/pull (delta — < 100 items)	< 800ms
GET /sync/pull (large delta — 100–500 items)	< 2500ms
POST /sync/push (up to 500 progress events)	< 1000ms
POST /notes (file upload accepted + queued)	< 500ms (excluding upload transfer time)
GET /notes/{note_id}/status (polling)	< 100ms (Redis-cached)
GET /sync/answers/{unit_id}	< 300ms
GET /progress/me	< 600ms
Admin dashboard	< 1500ms

9.2 Payload Optimisation

- Learning unit text_content fields MUST be compressed with gzip at the HTTP layer (Accept-Encoding: gzip in all client requests).
- Sync pull endpoint MUST use delta sync (only records changed since last_synced_at) — never return full data set.
- AI artefact payloads MUST omit correct_option_id in sync-pull responses — served separately via /sync/answers.

- All list endpoints MUST support cursor-based pagination (default page_size: 50, max: 200).
- Response envelopes MUST NOT include null fields (exclude_none=True in Pydantic models).
- Note metadata endpoints MUST NOT return file binary data — only paths and metadata.

9.3 Database Optimisation

- All foreign key columns MUST have indexes.
- Composite indexes MUST be created for all common query patterns (see Section 5 for index specifications).
- PostgreSQL connection pooling MUST be configured: min_connections=5, max_connections=20 (asyncpg pool via SQLAlchemy async).
- Heavy read queries (analytics, aggregated progress) MUST use PostgreSQL materialized views refreshed on schedule.
- N+1 query patterns MUST be eliminated: use SQLAlchemy selectinload/joinedload for all relationship loading.
- EXPLAIN ANALYZE MUST be run on all critical path queries during development; slow queries (> 50ms) MUST be optimised before deployment.

9.4 Caching Strategy

Cache Target	Strategy
Note processing status (polling)	Redis string cache, TTL 30s. Invalidated when status changes. Eliminates DB poll on every teacher status check.
Sync-pull delta query results	Redis hash keyed by user_id + since_timestamp. TTL 60s. Prevents repeated sync calls from re-querying DB.
User profile + permissions	Redis string cache per user_id. TTL 5 min. Invalidated on profile update.
Analytics aggregates	Redis cache for school dashboard queries. TTL 10 min. Background job warms cache.
AI artefact answers (post-completion)	Redis cache for /sync/answers per unit_id. TTL 1 hour. Prevents repeated DB joins.

9.5 Async & Concurrency

- FastAPI endpoints MUST use async def for all I/O-bound operations (DB queries, file reads, Redis).
- SQLAlchemy async sessions MUST be used throughout — no synchronous DB calls in async context.
- Celery worker concurrency MUST be set to min(4, CPU_count * 2) for the pilot deployment.
- Celery task priority queues: high (document parsing, artefact delivery), medium (question generation), low (analytics aggregation).

- Celery tasks MUST implement exponential backoff retry: max_retries=3, retry_backoff=True, retry_backoff_max=300.

10. File Management & Storage

Requirement	Specification
Accepted file types	.txt, .docx, text-based .pdf ONLY. Reject scanned/image PDFs at upload.
Maximum file size	10MB per upload. Enforce at nginx/reverse proxy layer AND FastAPI validation layer.
MIME validation	Validate actual MIME type using python-magic, not just file extension.
Content-addressed naming	Files stored as /data/notes/<school_id_hash>/<sha256_of_content>.<ext>. Prevents directory traversal. SHA-256 enables deduplication.
Deduplication	Before storing, compute SHA-256 of uploaded file. If file_hash exists for same school_id, return existing note_id (teacher uploaded same file twice).
Access control	File paths MUST NOT be directly exposed to clients. Serve files via presigned URLs (S3) or authenticated streaming endpoint. Android client never has a direct file URL.
Retention policy	Deleted notes: move file to /data/archive/ directory, retain for 90 days, then purge. Progress data retained indefinitely for research.
File size audit	background task weekly: audit total storage per school; alert system_admin if any school exceeds 1GB.

11. Background Job Specifications

11.1 Task: process_note

Attribute	Specification
Trigger	Enqueued by POST /notes immediately after file is saved
Queue	High priority
Steps	1. Parse document → 2. Chunk into learning units → 3. Store units in DB → 4. Enqueue generate_artefacts for each unit → 5. Update note status to 'processing'
Completion	Note status set to 'ready' when all child tasks complete

Failure handling	Note status set to 'failed'; error message stored; teacher notified via push notification (if FCM token available)
Estimated duration	< 30s for a typical 2000-word note (5–8 learning units)
Retry	max_retries=2; no retry on ParsingError (file is unprocessable); retry on DB errors

11.2 Task: generate_artefacts

Attribute	Specification
Trigger	Enqueued per learning unit by process_note task
Queue	Medium priority
Steps	1. Load unit text → 2. Run spaCy pipeline → 3. Generate MCQs → 4. Generate short answer Qs → 5. Generate summary → 6. Run QC checks → 7. Store artefacts in DB
Failure handling	Store partial artefacts; flag unit with generation_partial status; do not block note from becoming 'ready'
Estimated duration	2–10s per learning unit depending on unit length and NLP model load

11.3 Scheduled Jobs (Celery Beat)

Job Name	Schedule & Description
aggregate_analytics	Daily at 02:00 UTC. Compute school-level analytics aggregates from student_progress; store in analytics_events; refresh materialized views.
cleanup_expired_tokens	Every 6 hours. DELETE from refresh_tokens WHERE expires_at < now() OR is_revoked = TRUE AND created_at < now() - interval '7 days'.
sync_health_check	Hourly. Check for notes stuck in 'processing' status for > 30 minutes; re-enqueue process_note tasks for them; alert system_admin.
storage_audit	Weekly on Sunday 03:00 UTC. Compute total storage per school; log to analytics.
purge_archived_files	Monthly on 1st day 04:00 UTC. Delete files in /data/archive/ older than 90 days.

12. Error Handling & API Error Codes

All errors return a standardised envelope: { success: false, error: { code, message, details? } }. HTTP status codes are used semantically.

HTTP Status	Error Code	Scenario	Client Action	Example
400	VALIDATION_ERROR	Pydantic validation failed	Fix request payload per details field	Missing required field 'subject'
400	INVALID_FILE_TYPE	Uploaded file is not txt/docx/pdf	Show user correct formats	User uploads .ppt file
400	FILE_TOO_LARGE	File exceeds 10MB	Compress or split document	12MB PDF uploaded
400	IMAGE_PDF	PDF contains only scanned images	Ask teacher for text-based PDF	Scanned exam paper PDF
401	INVALID_CREDENTIALS	Wrong password/username	Show login error	Login failed
401	TOKEN_EXPIRED	Access token expired	Refresh token and retry	15 min passed
401	TOKEN_REVOKED	Refresh token revoked	Force re-login	Password was changed
403	FORBIDDEN	Insufficient role permissions	Show access denied	Student accessing admin
404	NOT_FOUND	Resource does not exist	Remove stale reference	Deleted note requested
409	DUPLICATE_FILE	Identical file already uploaded	Return existing note_id	Teacher re-uploads same file
422	PROCESSING_FAILED	Note could not be parsed	Show error to teacher	Corrupt docx file
429	RATE_LIMITED	Too many requests	Retry after Retry-After header	10+ failed logins
500	INTERNAL_ERROR	Unexpected server error	Retry with exponential backoff	DB connection lost
503	SERVICE_UNAVAILABLE	Worker queue overloaded	Retry after 30s	Celery queue full

13. Observability, Logging & Monitoring

13.1 Structured Logging

- All log output MUST be structured JSON (using structlog or loguru with JSON sink).
- Every HTTP request MUST be logged with: request_id (UUID injected by middleware), method, path, status_code, duration_ms, user_id (if authenticated), school_id.
- Every Celery task MUST log: task_name, task_id, note_id, status, duration_ms, error (on failure).
- Log levels: DEBUG (local dev only), INFO (request logs, task completion), WARNING (recoverable errors, rate limit hits), ERROR (exceptions, task failures), CRITICAL (DB down, queue full).
- Sensitive data (passwords, tokens, file content) MUST NEVER appear in logs.

13.2 Metrics

Metric	How Collected
API response time (P50, P95, P99) per endpoint	Prometheus middleware (starlette-prometheus)
Celery task duration and success/failure rate	Celery signals to Prometheus
PostgreSQL query duration	SQLAlchemy event listeners
Redis hit/miss rate	Redis INFO stats, polled by background job
Active sync sessions count	Counter incremented/decremented in Redis
Notes processed / failed per hour	Celery metrics + Prometheus
Error rate per endpoint (4xx, 5xx)	Prometheus middleware

13.3 Alerting

- ALERT: 5xx error rate > 1% over 5 minutes → page on-call (Grafana / Uptime Robot for pilot).
- ALERT: Celery queue depth > 100 tasks for > 10 minutes → scale workers.
- ALERT: DB connection pool exhausted → immediate investigation.
- ALERT: Note stuck in 'processing' for > 30 minutes → auto-retry + notify system_admin.
- ALERT: Disk usage > 80% → notify system_admin.

14. Testing Requirements

14.1 Unit Tests

- Every service function (note parsing, text chunking, question generation, scoring) MUST have unit tests with mocked DB.
- Test document parsing with: valid .txt, valid .docx, valid text PDF, image PDF (must fail gracefully), corrupted file (must fail gracefully), empty file (must fail gracefully).
- Test question generation with: short unit (< 100 words), typical unit (300 words), maximum unit (600 words), unit with no named entities.
- Test JWT logic: valid token, expired token, revoked refresh token, tampered token.

14.2 Integration Tests

- Test full note upload → processing → sync-pull cycle using httpx AsyncClient against test DB.
- Test delta sync: upload 3 notes, sync, upload 2 more notes, sync again — verify only 2 new notes returned.
- Test answer reveal endpoint: verify correct_option_id not present in sync-pull; verify it IS present after /sync/answers.
- Test role-based access: student attempting teacher endpoint returns 403; teacher attempting admin endpoint returns 403.

14.3 Performance Tests

- Simulate 50 concurrent sync-pull requests with 200-item deltas each — P99 must stay under 2500ms.
- Simulate 10 concurrent note uploads — verify all are processed without queue blockage within 5 minutes.
- Test DB query plans with EXPLAIN ANALYZE for all endpoints in the critical path.

14.4 Coverage Target

- Minimum 80% code coverage for all service and repository layers (pytest-cov).
- 100% coverage on authentication and file validation logic.
- CI pipeline MUST block merge if coverage drops below 80%.

15. Deployment & Infrastructure

15.1 Containerisation

Component	Docker Configuration
FastAPI App	Python 3.11-slim base; Uvicorn + Gunicorn entrypoint; non-root user; read-only filesystem except /tmp

Celery Worker	Same image as FastAPI app; CMD overridden to celery worker; mounted /data/notes volume
Celery Beat	Same image; CMD overridden to celery beat; single replica only
PostgreSQL	postgres:15-alpine; persistent named volume; automatic pg_dump backup via scheduled container
Redis	redis:7-alpine; append-only persistence (AOF); maxmemory-policy allkeys-lru
Nginx	nginx:alpine as reverse proxy; handles SSL termination, gzip compression, rate limiting at proxy layer

15.2 Environment Configuration

All environment-specific values MUST be configured via environment variables. A .env.example file with all required keys (no values) MUST be committed to the repository.

Variable	Purpose
DATABASE_URL	PostgreSQL async connection string
REDIS_URL	Redis connection string
JWT_SECRET_KEY	HS256 signing key (min 32 chars, random)
JWT_ALGORITHM	HS256
ACCESS_TOKEN_EXPIRE_MINUTES	15
REFRESH_TOKEN_EXPIRE_DAYS	30
STORAGE_BACKEND	'local' or 's3'
STORAGE_LOCAL_PATH	Absolute path to note storage directory
S3_BUCKET / S3_KEY / S3_SECRET	For S3-compatible storage
MAX_FILE_SIZE_MB	10
CELERY_WORKER_CONCURRENCY	4
SPACY_MODEL	en_core_web_sm
SENTENCE_TRANSFORMER_MODEL	paraphrase-MiniLM-L6-v2
SENTRY_DSN	Error tracking (optional for pilot)
CORS_ORIGINS	Comma-separated list of allowed origins
LOG_LEVEL	INFO

15.3 Pilot Deployment (Recommended)

- Platform: Railway.app or Render.com (free tier supports PostgreSQL + Redis + web service).
- Alternatively: single low-cost VPS (e.g. Hetzner CX11 €3.79/month) running Docker Compose.
- SSL: Let's Encrypt via Certbot (automated renewal).
- Domain: subdomain of project domain e.g. api.visiolearn.ug
- Backups: pg_dump to file daily; upload to Backblaze B2 (free tier 10GB).

16. Recommended Project Structure

The following directory structure is prescribed for the backend repository. Deviations must be documented and justified.

Path	Purpose
visiolearn-backend/	Project root
app/	Main application package
app/main.py	FastAPI application factory; middleware; router registration
app/config.py	Pydantic Settings model loading all env vars
app/database.py	SQLAlchemy async engine + session factory
app/models/	SQLAlchemy ORM model classes (one file per entity)
app/schemas/	Pydantic request/response schemas (one file per domain)
app/routers/	FastAPI routers: auth.py, users.py, notes.py, sync.py, progress.py, admin.py
app/services/	Business logic layer: auth_service.py, note_service.py, sync_service.py, ai_service.py, progress_service.py
app/repositories/	Database query layer (repository pattern): one file per model
app/workers/	Celery app + tasks: celery_app.py, note_tasks.py, ai_tasks.py, analytics_tasks.py
app/ai/	AI pipeline modules: parser.py, chunker.py, question_generator.py, summariser.py, scorer.py
app/storage/	Storage abstraction: base.py, local.py, s3.py — switchable via config
app/middleware/	Request ID injection, rate limiting, timing middleware
app/utils/	Shared utilities: security.py (JWT/hash), file_utils.py, pagination.py

app/exceptions.py	Custom exception classes with HTTP status codes
alembic/	Database migration files
tests/	pytest test suite mirroring app/ structure
tests/conftest.py	Fixtures: test DB, test client, authenticated users
Dockerfile	Production container definition
docker-compose.yml	Local development full stack
docker-compose.prod.yml	Production overrides
.env.example	Template for all required environment variables
pyproject.toml	Dependencies (poetry) + ruff + mypy config
README.md	Developer onboarding: setup, run, test, deploy

17. Acceptance Criteria

The backend is considered feature-complete and ready for pilot when ALL of the following conditions are met:

27. A teacher can register, log in, upload a .txt/.docx/.pdf lesson note, and the note transitions to 'ready' status within 5 minutes.
28. At least 3 MCQ questions and 1 summary are generated for every learning unit with >= 150 words.
29. Correct answers are NOT present in the sync-pull payload and ARE correctly returned by /sync/answers after completion is submitted.
30. A student can sync-pull their assigned content in < 800ms (P99) on a 3G-simulated network test.
31. A student's offline progress can be sync-pushed successfully after 48 hours of offline use, with no duplicate records in DB.
32. Delta sync is verified: syncing twice without any new content returns an empty payload in < 150ms.
33. A teacher can mark an AI artefact as unapproved, and it is absent from subsequent student sync-pulls.
34. All RBAC rules defined in Section 8.2 are enforced — verified by integration tests.
35. Rate limiting on /auth/login blocks accounts with 10+ failed attempts within 15 minutes.
36. All unit and integration tests pass with >= 80% coverage.
37. POST /notes with an image-only PDF returns a 422 INVALID_FILE error.
38. The system handles 50 concurrent sync-pull requests without error (load test verified).
39. Docker Compose starts the full stack (API + DB + Redis + Worker) with docker compose up on a fresh machine.
40. OpenAPI documentation is accessible at /docs and accurately reflects all endpoints.

18. Development Phases & Priorities

Phase	Sprint	Deliverables	Priority
Phase 1: Core API	Sprint 1–2 (Weeks 1–4)	Project scaffold, DB setup, Alembic migrations, user auth (register/login/refresh/logout), school management, note upload (no processing), basic file storage	CRITICAL
Phase 2: Processing	Sprint 3–4 (Weeks 5–8)	Celery setup, document parser (txt/docx/pdf), text chunker, learning unit storage, note status management, teacher polling endpoint	CRITICAL
Phase 3: AI Pipeline	Sprint 5–6 (Weeks 9–12)	spaCy integration, MCQ generator, short-answer generator, summariser, QC pipeline, artefact storage, teacher review/approval endpoints	CRITICAL
Phase 4: Sync	Sprint 7–8 (Weeks 13–16)	Delta sync-pull endpoint, progress sync-push, answer reveal endpoint, note assignment system, sync status endpoint	CRITICAL
Phase 5: Analytics	Sprint 9 (Weeks 17–18)	Progress aggregation, school analytics endpoints, anonymised CSV export, materialized views, scheduled aggregation job	HIGH
Phase 6: Admin & Ops	Sprint 10 (Weeks 19–20)	Admin dashboard endpoints, bulk user import, error management, logging, monitoring setup, load testing, pilot deployment	HIGH
Phase 7: Hardening	Sprint 11 (Weeks 21–22)	Security audit, penetration testing of auth flows, performance profiling, slow query elimination, full test coverage, documentation	HIGH

19. Open Questions & Decisions Required

#	Question	Impact
1	Will Makerere University's Luganda Neural TTS be integrated server-side (backend pre-generates audio files) or client-side (Android uses it locally)? Server-side increases storage and processing cost but improves consistency.	Storage, bandwidth, AI pipeline design

2	What is the push notification strategy for teacher alerts (note processing complete)? Options: Firebase Cloud Messaging (requires Google Play Services — may not be available on all basic phones), polling-only, or in-app badge on next open.	Notification module, FCM dependency
3	Should teachers be able to edit/correct AI-generated questions (not just approve/reject)? This adds a teacher content editor to the API surface.	API scope, artefact versioning
4	Are there plans to integrate with the MoH DHIS2 platform during the pilot? This determines the format of the analytics export endpoint.	Analytics schema design
5	What is the maximum expected concurrent user count for the pilot? (Estimated 50 students + 10 teachers) — determines worker and DB sizing.	Infrastructure sizing
6	Should student usernames be assigned by school admin or self-registered? Students without email addresses need a simple onboarding flow.	Auth flow complexity

20. Reference Documents

- Original Research Proposal: VisioLearn — Interactive Audio Learning Platform for Visually Impaired Students (Uganda), UTAMU, 2026.
- FastAPI Official Documentation: https://fastapi.tiangolo.com
- SQLAlchemy 2.0 Async Documentation: https://docs.sqlalchemy.org/en/20/orm/extensions/asyncio.html
- Celery Documentation: https://docs.celeryq.dev
- spaCy Documentation: https://spacy.io/usage
- PDFMiner.six Documentation: https://pdfminersix.readthedocs.io
- python-docx Documentation: https://python-docx.readthedocs.io
- sentence-transformers: https://www.sbert.net
- PostgreSQL 15 Documentation: https://www.postgresql.org/docs/15/
- OWASP API Security Top 10: https://owasp.org/www-project-api-security/

— END OF DOCUMENT —

VisioLearn Backend PRD v1.0 | Prepared March 2026